

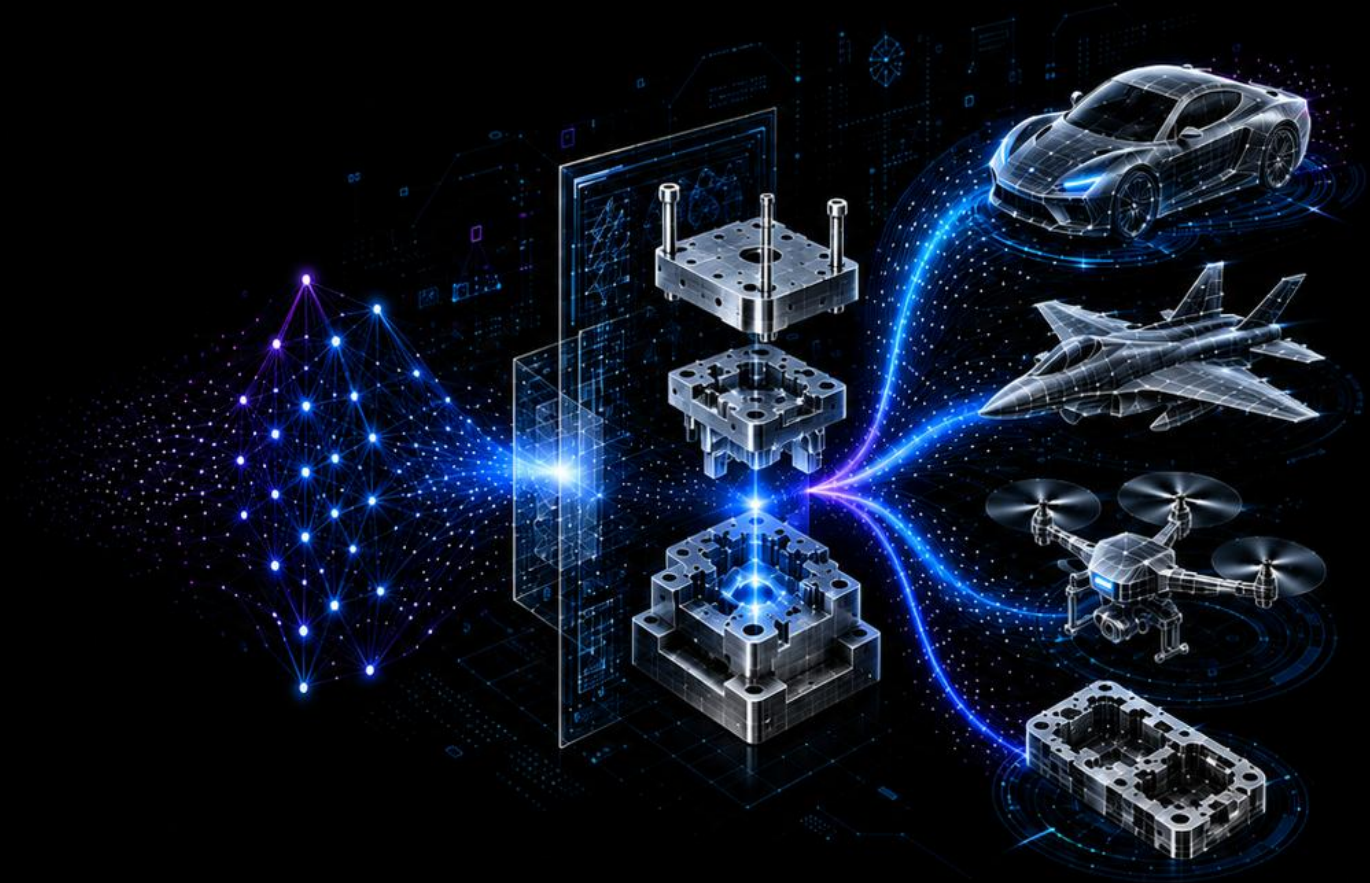
AI 스터디

RNN/LSTM/ GRU

안동균

(지도교수: 양선웅)

한양대학교 ERICA 로봇공학과



Intelligent Design
for Engineering
Applications Lab

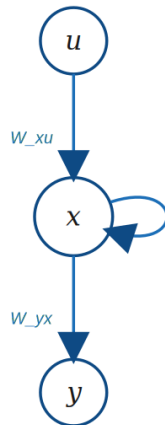
RNN(Recurrent Neural Network)

시계열 데이터(sequence data): 시간에 따라 얻어진 데이터 → 순서가 중요한 데이터
따라서 시계열 데이터는 시간에 영향을 받고 이전 시점의 데이터가 중요하다.

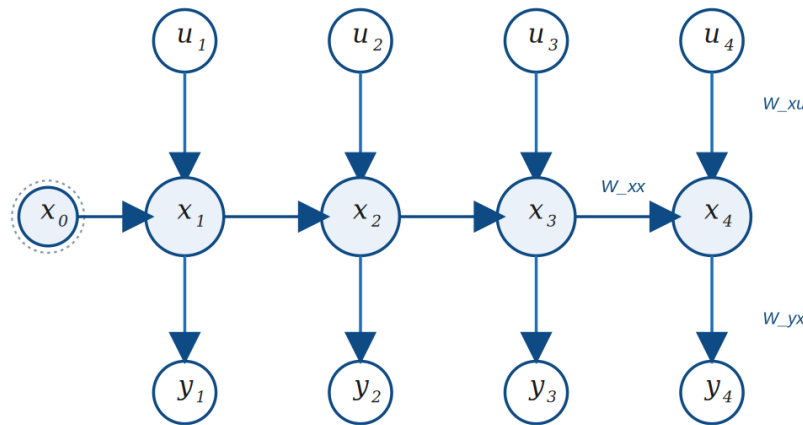
RNN(Recurrent Neural Network)은 시간적으로 연속성이 있는 데이터를 처리하기 위한 인공 신경망
기존 네트워크와 다른점 → 기억을 갖는다=이전 시점의 정보를 반영한다 (이전 은닉층이 현재 은닉층의 입력이 되면서 반복되는 순환 구조를 갖는다)

CNN이 공간(이미지)의 지역 패턴을 뽑았다면, RNN 계열은 시간(순서)의 패턴을 뽑는다.

□□□□ □□
(□□ □□)

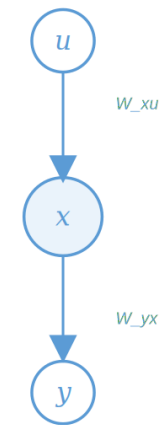


□□□□ □□ RNN
state-space model unfolded in time



RNN

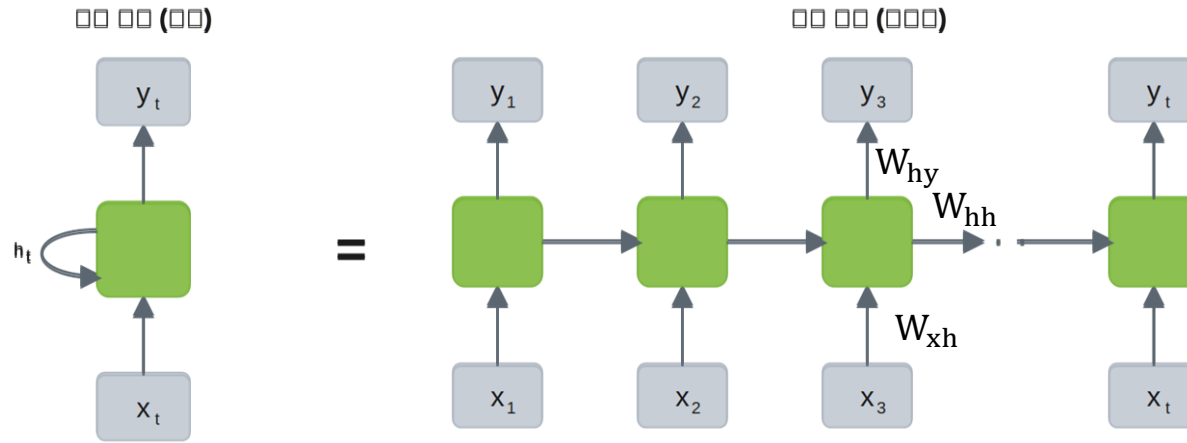
ANN
(□□ □□ · □ □□)



ANN

RNN 구조와 수식

Part 1 RNN

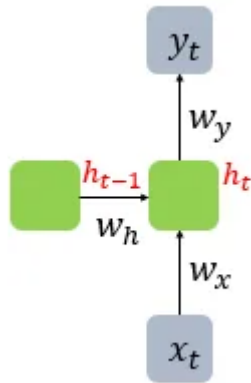


은닉레이어의 상태 $h_t = f(W_{xh} \cdot x_t + \text{이전시점 정보})$
 출력값 $y_t = g(W_{hy} \cdot h_t)$
 →이전 시점의 정보를 반영하여 더 나은 예측을 하자

Part 2 LSTM

Part 3 GRU

Part 4 실습 · 정리



은닉층

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

출력층

$$y_t = f(W_{y} \cdot h_t + b) \text{ } f \text{는 소프트맥스함수}$$

가중치 공유

W_{xh} , W_{hh} , W_{hy} 는 모든 시점에서 동일
 ↔CNN은 이미지 구역별로 같은 weight를 공유한다면
 RNN은 매 시점 같은 weight를 공유한다

학습대상 파라미터: W_{xh} , W_{hh} , W_{hy}
 Loss가 최소가 되도록하는 W 를 구하는것이 목표

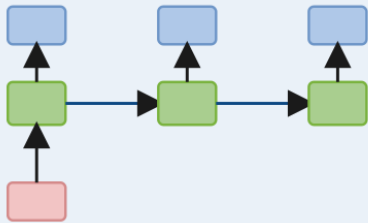
h_{t-2} h_{t-1} h_t 이전 시점의 정보 모두 이용하여 y_t 예상

핵심: 펼친 그림에서 셀이 여러 개로 보여도 사실은 **같은 셀 하나**다. 가중치를 공유하기 때문에 시점이 아무리 길어도 파라미터 수는 그대로.
 매 타임스텝, 직전 상태와 현재 입력으로 새 상태를 만든다

RNN 구조 다양성

입력과 출력의 개수에 따라 여러 형태로 쓰인다

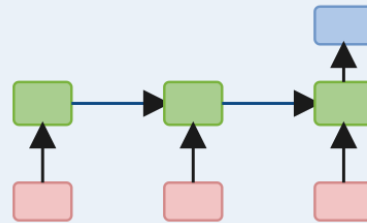
one-to-many



단일 입력 → 순차 출력

예: 생성 (이미지 캡셔닝)

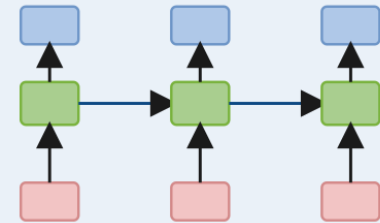
many-to-one



순차 입력 → 단일 출력

예: 예측 (감성 분류)

many-to-many



순차 입력 → 순차 출력

예: 번역, 품사 태깅(POS)

특수 케이스 — Sequence to Sequence (seq2seq)

many-to-many를 many-to-one(Encoder) + one-to-many(Decoder) 로 이은 구조.

Encoder가 입력 문장을 **요약 벡터**로 압축 → Decoder가 그걸 보고 출력 생성. 예: 영어→한글 번역.

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

RNN 학습 ① 순전파 (Forward)

① 순전파 (Loss 계산)

② 역전파 (Gradient 계산)

③ 파라미터 업데이트

$$h_t = \tanh(Wxh \cdot x_t + Whh \cdot h_{t-1})$$

각 weight 곱하고 더한 값을 -1~1로 정규화한 hidden state
$$\hat{y}_t = \text{softmax}(Why \cdot h_t)$$

hidden state에 weight 곱해 0~1 확률로 정규화한 예측값
$$L_t = y_t \text{ 와 } \hat{y}_t \text{ 의 차이 (loss)}$$

h_t = 은닉 상태

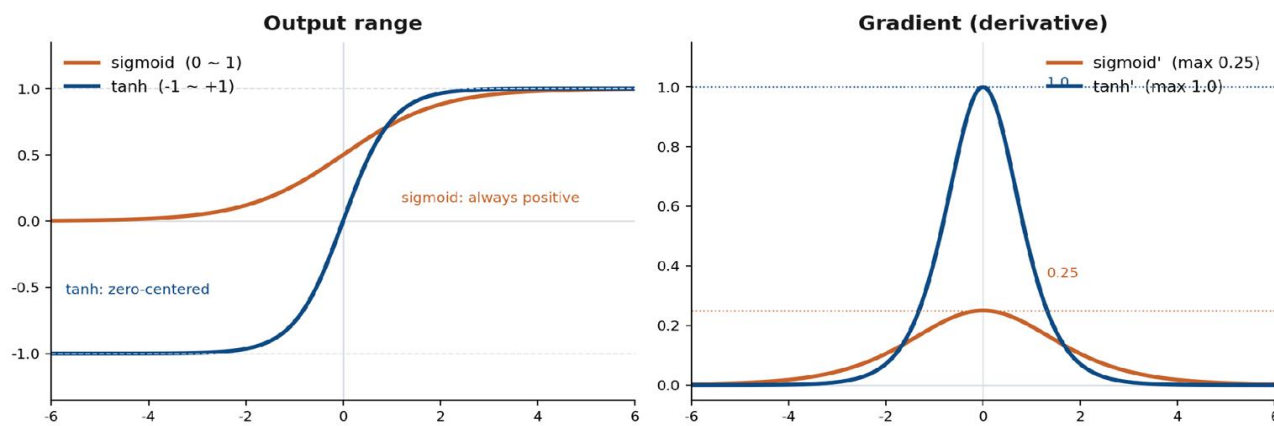
과거 정보를 담아 다음 시점으로 전달

$\hat{y}_t$ = 예측값

softmax로 클래스별 확률 출력

L_t = 손실

정답 y_t 와 예측 $\hat{y}_t$ 의 차이



은닉상태를 구하는 활성화 함수는
시그모이드 대신 tanh사용

학습 3단계: 순전파로 Loss를 구하고 → 역전파로 gradient → 업데이트, 이 순환을 반복.

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

Part 1 RNN

Backpropagation Through Time: W 가 Loss에 미친 영향(gradient) 계산
gradient는 chain rule로 계산. Why 는 간단하지만, Whh 는 시간을 거슬러 올라가며 합산되므로 시점이 길어지면 복잡해짐.

Why 의 gradient (한 시점)

$$\partial Loss / \partial Why = (\partial L_t / \partial \hat{y}_t) \times (\partial \hat{y}_t / \partial (Why \cdot h_t)) \times (\partial (Why \cdot h_t) / \partial Why)$$

Part 2 LSTM

Whh 의 gradient (모든 과거 시점의 영향을 합산)

$$\partial Loss / \partial Whh = T_3 + T_2 + T_1 \quad (t=3 \text{ 예시})$$

$T_3 : (\partial Loss / \partial \hat{y}_t) \times (\partial \hat{y}_t / \partial h_3) \times (\partial h_3 / \partial Whh)$	시점 3에서의 영향
$T_2 : \dots \times (\partial h_3 / \partial h_2) \times (\partial h_2 / \partial Whh)$	시점 3→2로 전해진 영향
$T_1 : \dots \times (\partial h_3 / \partial h_2) \times (\partial h_2 / \partial h_1) \times (\partial h_1 / \partial Whh)$	시점 2→1로 전해진 영향

Part 3 GRU

왜 시점마다 더하나?

Whh 는 모든 시점에서 공유되므로, 각 시점의 h_t 를 거쳐 Loss에 영향을 준다.

그 경로들을 chain rule로 펼쳐 전부 더한 것이 Whh 의 최종 gradient. **이 곱셈의 사슬이 길어지면 → 기울기 소실.**

Part 4 실습 · 정리

과거로 갈수록 $\partial h_t / \partial h_{t-1}$ (0~1)이 계속 곱해져 0에 수렴 → 먼 과거가 학습되지 않는다.

RNN 학습 ③ 파라미터 업데이트

Part 1
RNN

계산한 gradient로 W 를 더 좋은 값으로 갱신 (Gradient Descent)

학습대상 파라미터(W_{xh}, W_{hh}, W_{hy}) → 역전파를 통해 gradient 계산

η (learning rate): 기여도를 얼마나 반영할지 (예: $\eta = 0.01$)

방향

Loss를 줄이는 쪽 (gradient의 반대 방향)

크기

gradient가 큰 파라미터일수록 많이 갱신

학습률 η

한 번에 얼마나 이동할지 조절

Part 2
LSTM

세 파라미터 각각의 기여도 = Loss에 대한 편미분

W_{xh}

$\partial \text{Loss} / \partial W_{xh}$

입력 반영

W_{hh}

$\partial \text{Loss} / \partial W_{hh}$

과거 반영

W_{hy}

$\partial \text{Loss} / \partial W_{hy}$

출력 예측

Part 3
GRU

학습 = 이 3단계의 반복 → 앞서 배운 코드들과 동일

① 순전파(Loss) → ② 역전파(gradient) → ③ 업데이트 ... 다시 ①로.

Loss가 더 이상 줄지 않을 때까지 반복하며 $W_{xh} \cdot W_{hh} \cdot W_{hy}$ 를 최적값으로 수렴시킨다.

Part 4
실습 · 정리

방향은 Loss를 줄이는 쪽, 크기는 gradient만큼 — 이것이 경사하강법(Gradient Descent).

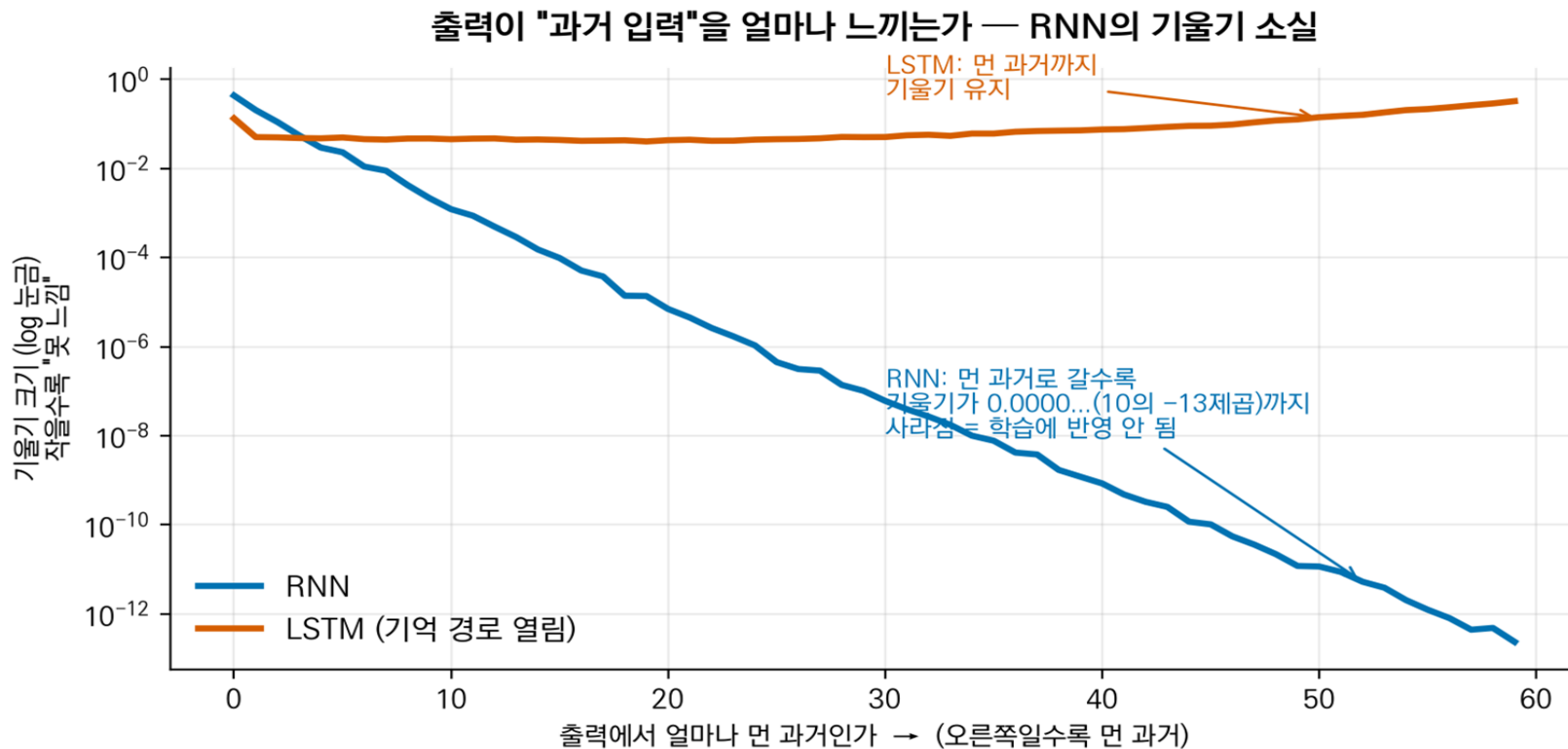
RNN의 한계 (실측) — 장기 의존성 문제

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리



RNN이 갖는 한계점: exploding / vanishing gradient

- 은닉상태 h_t 에는 W_{hh} 가 계속 곱해지는데
- 곱해지는 값이 1보다 크면 발산
 - 곱해지는 값이 1보다 작으면 0으로 수렴

출력에서 먼 과거일수록 RNN의 기울기가 0에 가깝게 사라진다(10^{-13}). LSTM은 유지. 랜덤 초기화 30회 평균.

순환신경망 개선 모델: LSTM

Part 1
RNN

Part 2
LSTM

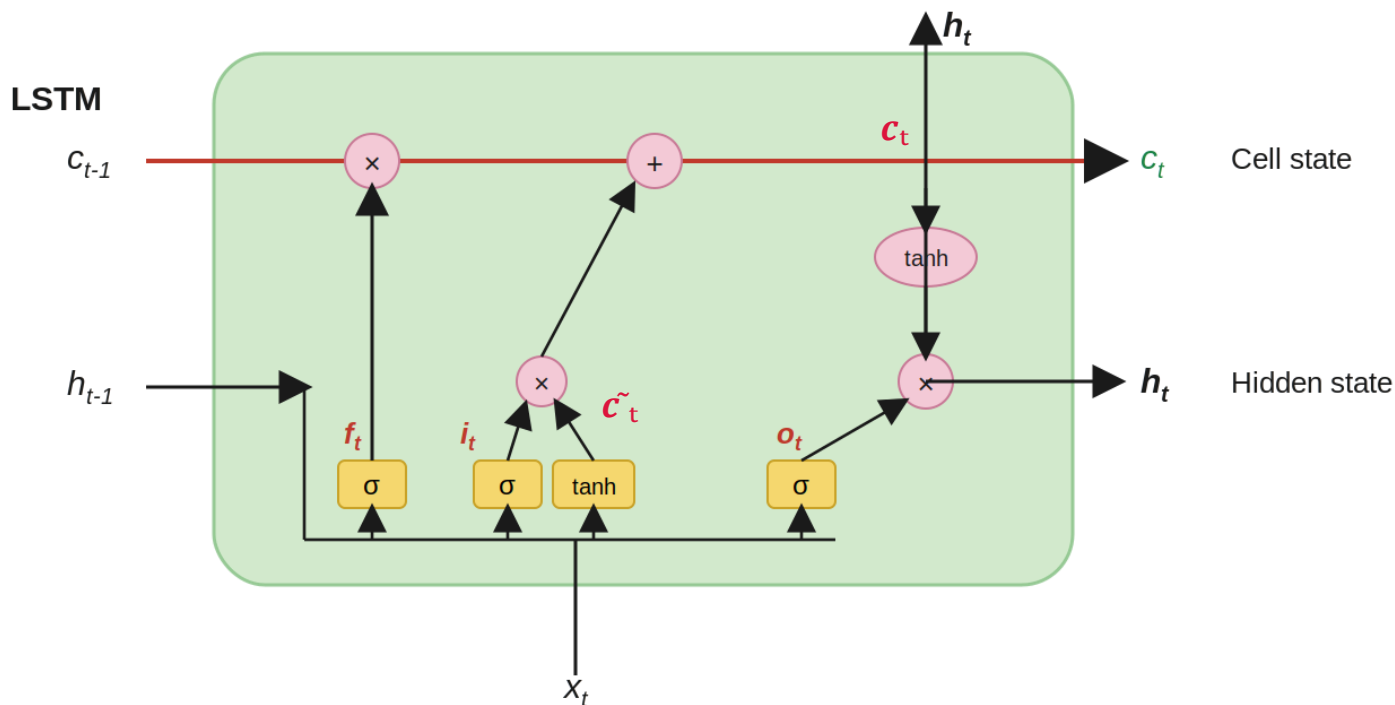
Part 3
GRU

Part 4
실습 · 정리

Long Short Term Memory 는 장기 의존성 문제를 완화한 RNN 개선 모델

Cell state (c_t) 구조를 제안하고, 세 가지 gate를 추가한 구조

세 gate: Forget (f_t) · Input (i_t) · Output (o_t)



$$f_t = \sigma(Wx_{hf} \cdot x_t + Wh_{hf} \cdot h_t^{-1} + bf)$$

$$i_t = \sigma(Wx_{hi} \cdot x_t + Wh_{hi} \cdot h_t^{-1} + bi)$$

$$o_t = \sigma(Wx_{ho} \cdot x_t + Wh_{ho} \cdot h_t^{-1} + bo)$$

⊗ 는 합성곱

셀 상태 c_t (장기 기억)와 은닉 상태 h_t (단기 기억)를 분리하고, 3개 게이트가 정보 흐름을 조절한다.

LSTM 계산 3단계

게이트 → 셀 상태 → 은닉 상태 순서로 매 시점 갱신

Part 1
RNN

1. Gate 계산하기

Forget (f_t) · Input (i_t) · Output (o_t)
각 게이트를 σ 로 계산



Part 2
LSTM

2. Cell state (c_t) 업데이트

$c_t = f_t \otimes c_{t-1} + i_t \otimes \tilde{c}_t$
과거는 지우고 새 정보는 담는다



Part 3
GRU

3. Hidden state (h_t) 업데이트

$h_t = o_t \otimes \tanh(c_t)$
내보낼 정보만 골라 h_t 생성

Part 4
실습 · 정리

이 3단계를 매 시점 t 마다 반복 — 게이트는 $\sigma(0\sim1)$, 후보와 셀은 $\tanh(-1\sim1)$ 로 계산된다.

셀 상태 c_t 는 장기 기억 통로, 은닉 상태 h_t 는 매 시점 출력으로 꺼내 쓰는 단기 기억.

① Gate 계산 — Forget · Input · Output

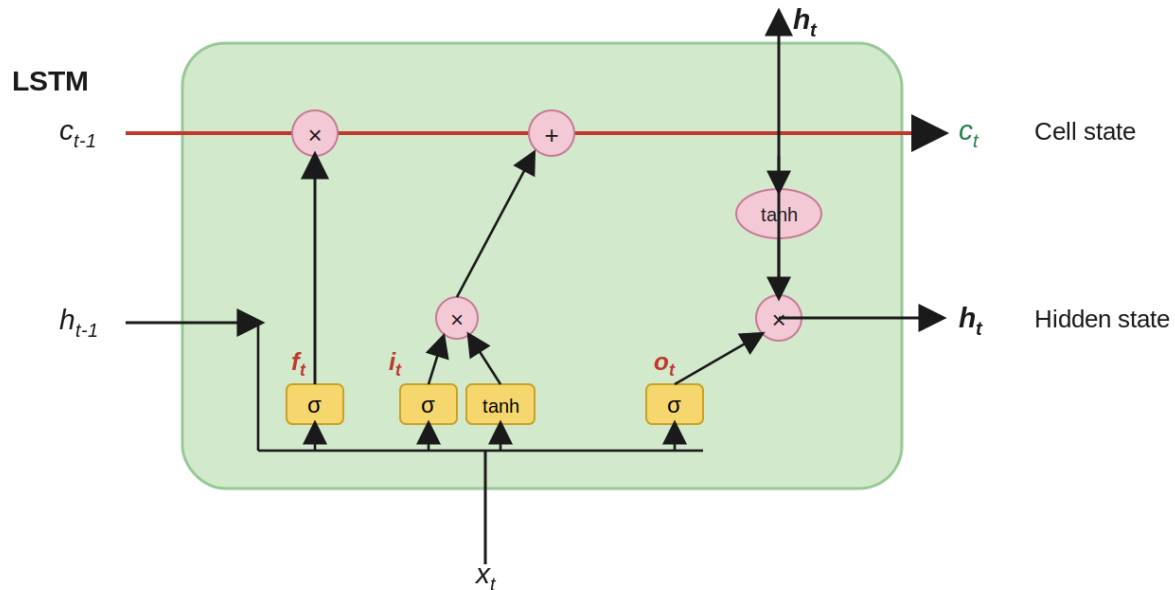
Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

Gate = $\sigma(Wx_h \cdot x_t + Wh_h \cdot h_{t-1} + b)$, 0~1 사이 값을 갖는 벡터(가중치 역할)



1 σ = 시그모이드
최종 출력이 0~1 → '얼마나 통과시킬지' 정하는 밸브

2 Forget f_t
과거 cell state에서 지울 정보의 가중치

3 Input i_t
cell state에 새로 담을 정보의 가중치

4 Output o_t
hidden state로 내보낼 정보의 가중치

5 게이트마다 다른 $W(W_{xh}, W_{hh})$
같은 (x_t, h_{t-1}) 이지만 W 가 달라 서로 다른 값

$$\begin{aligned} o_t &= \sigma(Wx_{ho} \cdot x_t + Wh_{ho} \cdot h_{t-1} + b_o) i_t \\ &= \sigma(Wx_{hi} \cdot x_t + Wh_{hi} \cdot h_{t-1} + b_i) f_t \\ &= \sigma(Wx_{hf} \cdot x_t + Wh_{hf} \cdot h_{t-1} + b_f) \end{aligned}$$

게이트는 벡터(1차원 행렬)다 — 예: $f_t = [0, 0.5, 1, \dots]$

② Cell state (c_t) 업데이트

Part 1
RNN

$$\tilde{c}_t = \tanh(Wxhg \cdot x_t + Whhg \cdot h_{t-1} + bg) \leftarrow \text{임시 cell state (현재 정보 요약, -1~1)}$$
$$c_t = f_t \otimes c_{t-1} + i_t \otimes \tilde{c}_t \leftarrow \text{과거 정보 + 새 정보}$$

Part 2
LSTM

숫자로 보는 forget 게이트: 0이면 지우고, 1이면 그대로 유지

f _t	0	0.2	0.9	0.1	1	0.2	0.1	0.9	0.4	0.1
c _{t-1}	-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7
f _t ⊗c _{t-1}	0	0.02	0.45	0.07	0.9	-0.06	-0.11	1.8	0.24	0.07

→ 첫 원소는 f_t=0 이라 0으로 완전히 지워지고, 다섯째는 f_t=1 이라 c_{t-1} 값 0.9가 그대로 유지된다.

Part 3
GRU

i _t ⊗c̃ _t	-0.01	0	0.48	0.04	0.72	0.07	0.4	1.89	0.12	0.36
c _t	-0.01	0.02	0.93	0.11	1.62	0.01	0.29	3.69	0.36	0.43

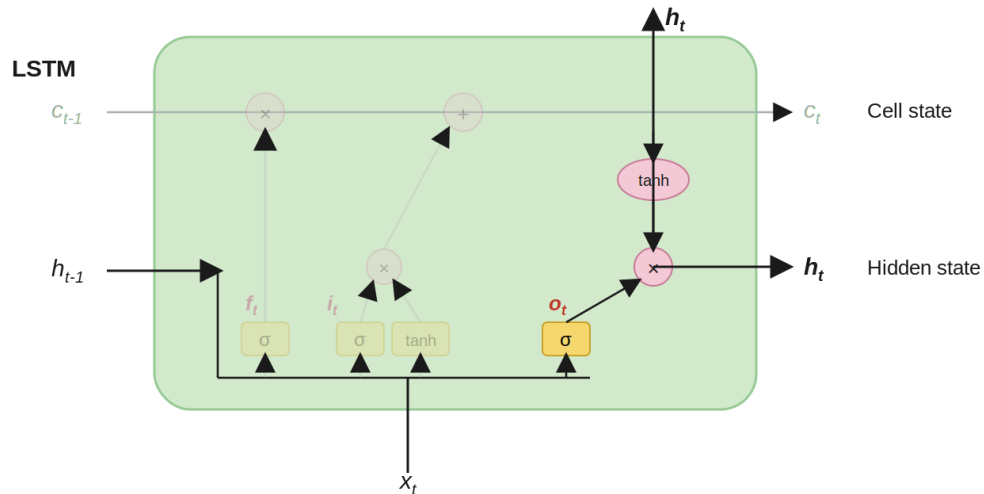
불필요한 정보는 잊고(f_t⊗c_{t-1}), 새 정보는 더해서(i_t⊗c̃_t) 새 cell state c_t를 완성.

덧셈(+)으로 셀 상태를 갱신하기 때문에 기울기가 소실 없이 멀리까지 흐른다.

Part 4
실습 · 정리

③ Hidden state (h_t) 업데이트

$h_t = o_t \otimes \tanh(c_t)$ — 완성된 셀 상태에서 내보낼 것만 고른다



$$o_t = \sigma(Wxho \cdot x_t + Whho \cdot h_{t-1} + bo)$$

$$h_t = o_t \otimes \tanh(c_t)$$

숫자로 보는 출력

o_t	0.5	0.4	0.1	0.9	0.2	0.3
c_t	-0.01	0.02	0.93	0.11	1.62	0.01
$\tanh(c_t)$	-0.01	0.02	0.73	0.11	0.92	0.01
h_t	0	0.08	0.07	0.1	0.18	0.0

- ① 셀 상태 c_t 를 $\tanh$ 로 $-1 \sim 1$ 로 누르고
 - ② 출력 게이트 $o_t(0 \sim 1)$ 를 곱해 걸러낸다
- 지금 내보낼 정보만 h_t 에 담긴다

h_t 는 출력층으로도, 다음 시점으로도 전달 — PyTorch의 out, (h, c) 중 h가 바로 이것.

Part 1
RNN

Part 2
LSTM

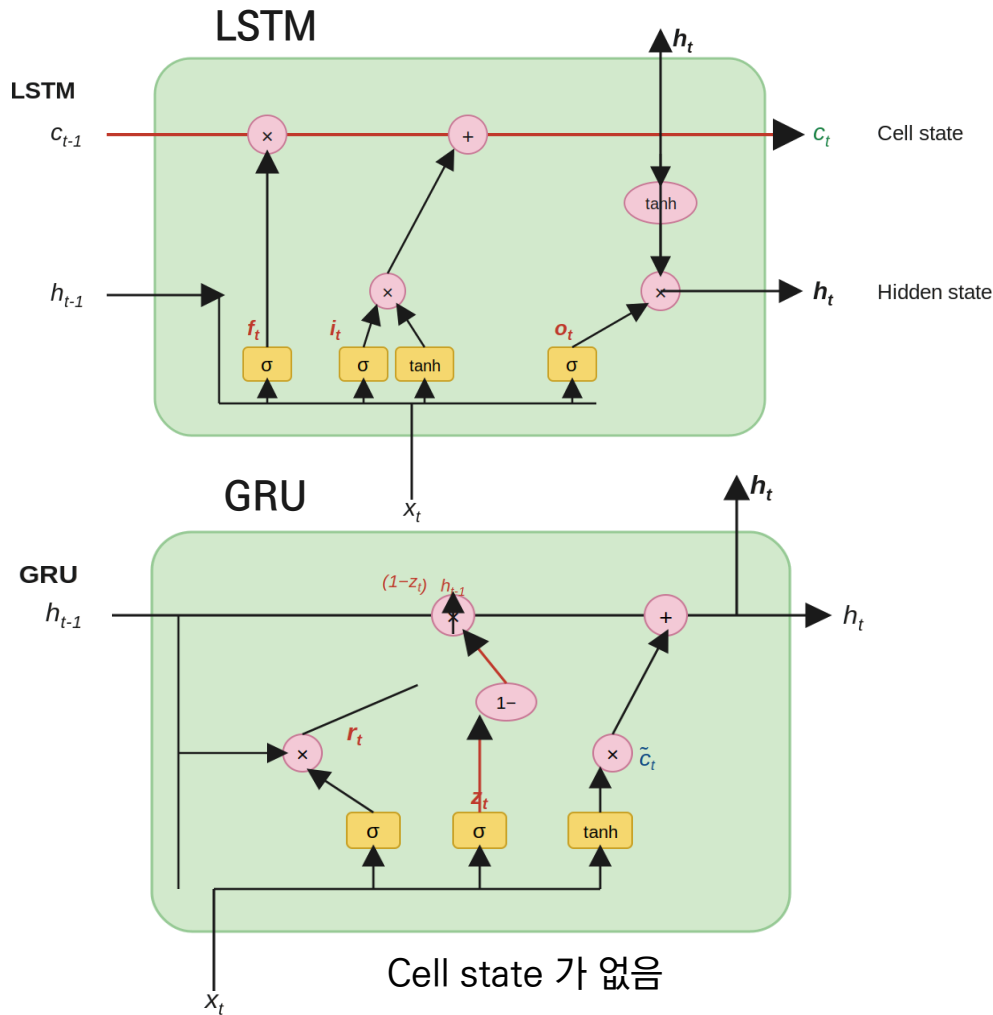
Part 3
GRU

Part 4
실습 · 정리

LSTM 구조 개선 모델: GRU

GRU는 LSTM 구조를 간단하게 개선하여 파라미터 개수를 줄인 모델

- forget · input gate → **update gate (z_t)** 로 통합
- output gate 제거 + **reset gate (r_t)** 정의
- cell state + hidden state → **hidden state 하나로 통합** (셀 상태 없음)



reset r_t : 과거 h_{t-1} 을 얼마나 무시하고

후보 $\tilde{c}_t$ 를 만들지 조절

update z_t : 과거와 새 후보를 섞는 비율

(z 하나가 forget+input 역할)

$\tilde{c}_t$

$= \tanh(Wxhg \cdot x_t + Whhg \cdot (r_t \otimes h_{t-1}) + bhg)$ 임시 **cell state** z_t

$= \sigma(Wxhz \cdot x_t + Whhz \cdot h_{t-1} + bhz)$ **Update gate** r_t

$= \sigma(Wxhr \cdot x_t + Whhr \cdot h_{t-1} + bhr)$ **Reset gate**

최종 hidden state (= cell state 통합)

$$h_t = (1 - z_t) \otimes h_{t-1} + z_t \otimes \tilde{c}_t$$

($1 - z_t$) = forget 역할 · z_t = input 역할

게이트 3→2개(reset·update), 상태 2→1개(hidden) → LSTM보다 가볍고 학습이 빠르다.

z 하나로 '과거 유지($1-z$)'와 '새 정보 반영(z)'을 동시에 조절 → LSTM의 두 게이트를 대체.

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

RNN vs LSTM vs GRU

Part 1 RNN

	RNN	LSTM	GRU
게이트 수	0	3	2

상태(hidden state)

h

h, C (셀 상태)

h

Part 2 LSTM

장기 의존성	약함	강함	강함
--------	----	----	----

파라미터 / 속도

적음 / 빠름

많음 / 느림

중간 / 빠름

반환값 (PyTorch)

out, h

out, (h, c)

out, h

Part 3 GRU

언제 쓰나

짧은 시퀀스

성능 우선 · 긴 시퀀스

성능 비슷 · 경량

Part 4 실습 · 정리

실무 결론: 성능은 LSTM $\approx$ GRU. 하지만 GRU가 LSTM보다 파라미터가 훨씬 작기 때문에 training time이 절약됨
→ 그럼 GRU가 무조건 성능이 좋은가? → Task에 따라 천차만별
따라서 둘 다 돌려보고 데이터·태스크에 맞게 고른다.

But LSTM과 GRU가 RNN보다는 성능이 좋음을 보장한다

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

```
# 리뷰(정수 시퀀스) → 임베딩 → RNN → 긍정/부정 2분류

class Net(nn.Module):
    def __init__(self):
        self.em = nn.Embedding(vocab_size, 100) # 단어 → 100차원 벡터
        self.rnn = RNNCell_Encoder(100, 300) # RNN 인코더(은닉 300)
        self.fc1 = nn.Linear(300, 256)
        self.fc2 = nn.Linear(256, 2) # 2 클래스(긍/부정)
    def forward(self, x):
        x = self.rnn(self.em(x)) # 임베딩 후 RNN 인코딩
        return self.fc2(F.relu(self.fc1(x)))

loss_fn = nn.CrossEntropyLoss() # 분류 → CrossEntropy
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- 리뷰를 정수로 인코딩 → 임베딩 → RNN이 순서대로 읽어 문장 벡터를 만든다.
- 마지막 은닉 상태 → FC → 긍정/부정 2클래스, 손실은 CrossEntropy.
- 결과: 정확도 약 50% → 긴 리뷰(200단어)에서 RNN의 한계가 드러남.

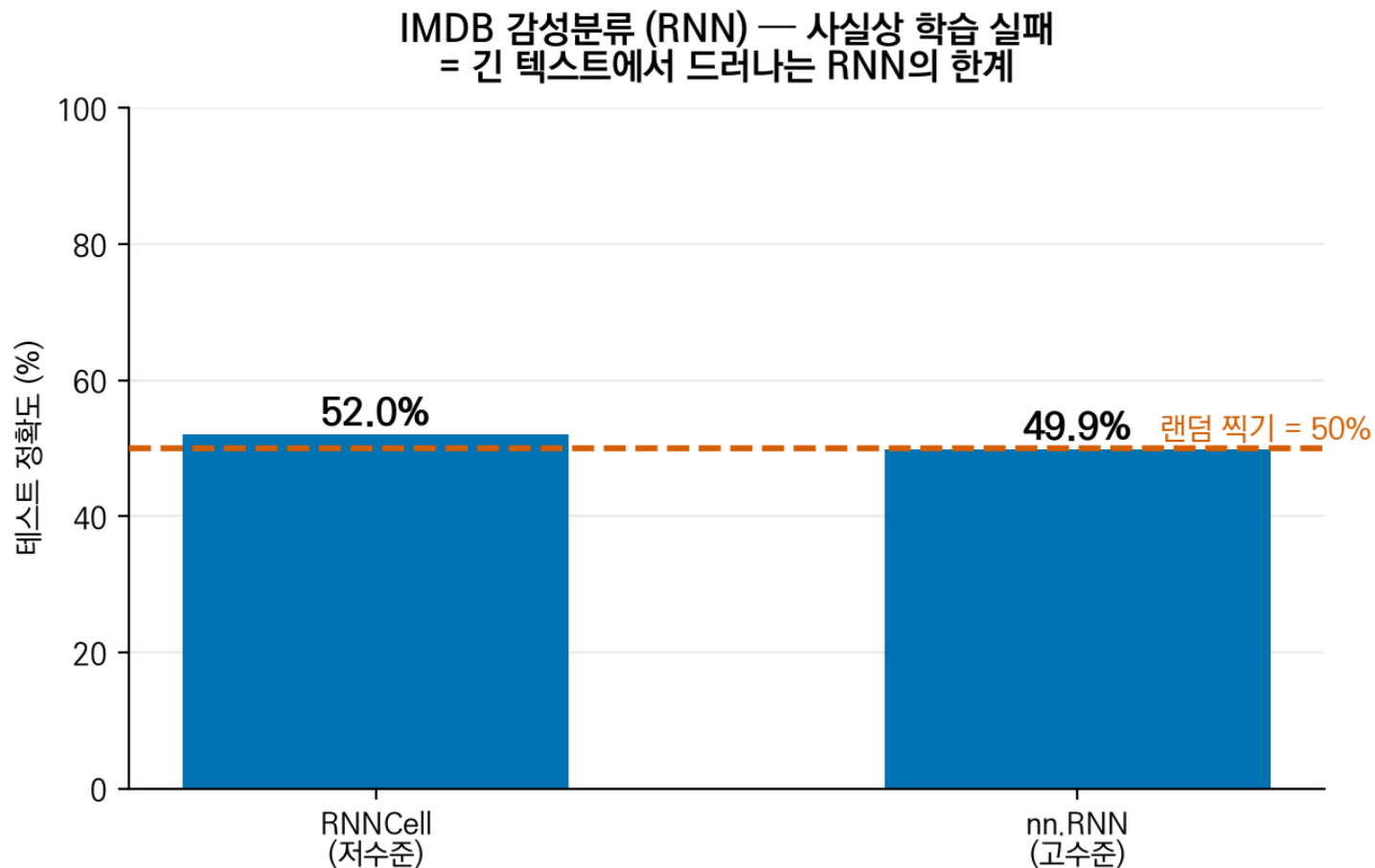
실습① IMDB 결과 — RNN의 한계 실증

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리



200단어 리뷰에서 RNN 두 버전 모두 약 50%(랜덤 수준). 긴 시퀀스 학습 실패를 실제로 보여준다.

Part 1 RNN

Part 2 LSTM

Part 3 GRU

Part 4 실습 · 정리

```
# 28x28 이미지를 28스텝 × 28차원 시퀀스로 보고 분류

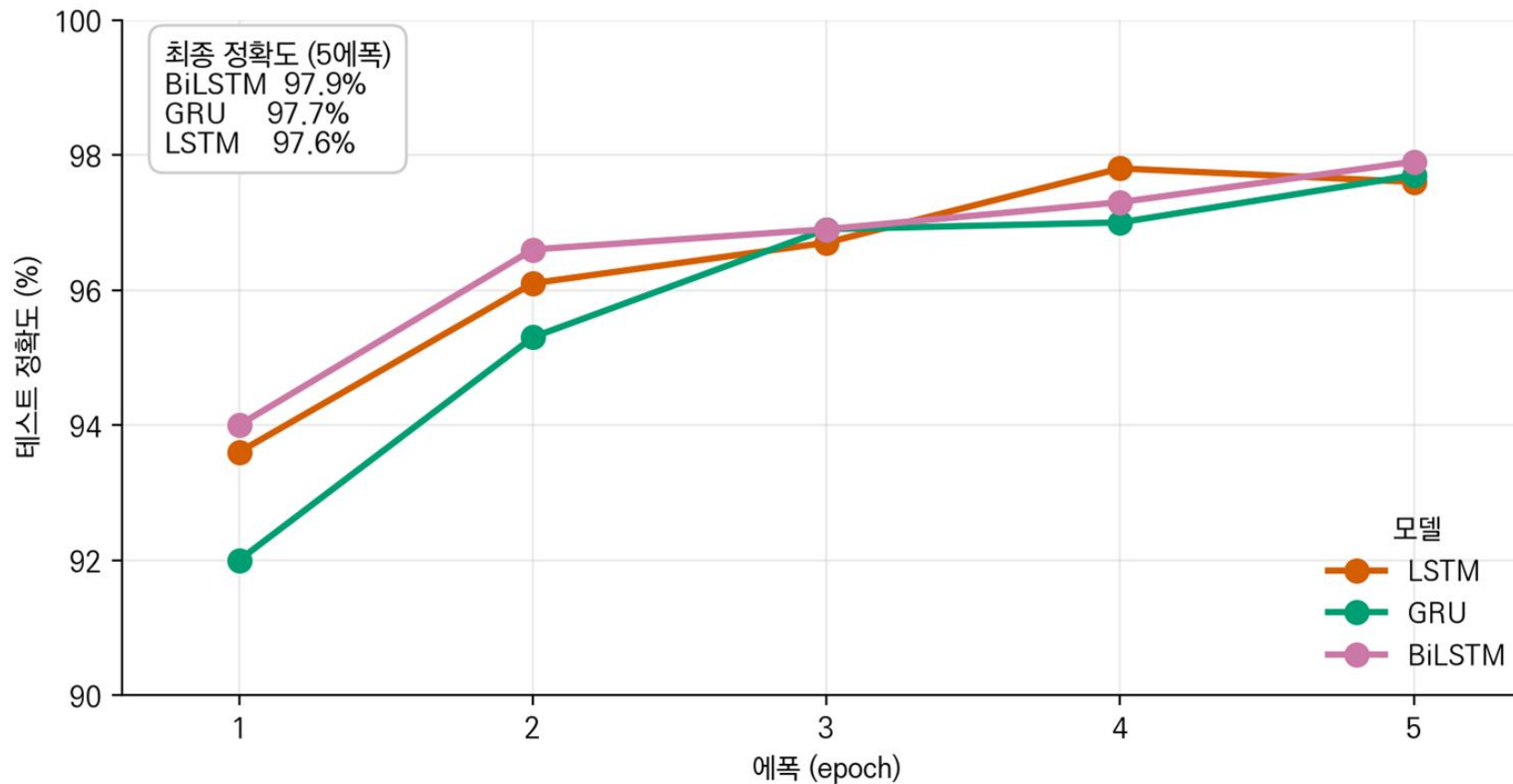
class LSTMModel(nn.Module):
    def __init__(self):
        self.lstm = nn.LSTM(28, 128, batch_first=True)
        self.fc = nn.Linear(128, 10) # 숫자 10클래스
    def forward(self, x): # x: (batch, 28, 28)
        out, (h, c) = self.lstm(x, (h0, c0)) # LSTM은 (h, c) 반환
        return self.fc(out[:, -1, :]) # 마지막 스텝 → 분류

# GRU : nn.LSTM → nn.GRU, (h0,c0) → h0 하나만
# BiLSTM : bidirectional=True, fc 입력 = 128 * 2
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3) # SGD→Adam이 핵심
```

- 이미지를 한 행씩(28스텝) 읽는 시퀀스로 변환해 순환신경망에 입력.
- LSTM → GRU → BiLSTM 은 한두 줄 차이 (셀 상태 유무, 양방향 여부).
- SGD를 Adam으로 바꾸니 5에폭에 97~98% (SGD는 학습이 안 됨).

실습② MNIST 결과 — LSTM · GRU · BiLSTM

MNIST 시퀀스 분류 — LSTM · GRU · BiLSTM 정확도 (5에폭)



세 모델 모두 5에폭에 97~98%로 잘 학습. 양방향 BiLSTM이 근소 우위(교과서 속 예제코드)

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

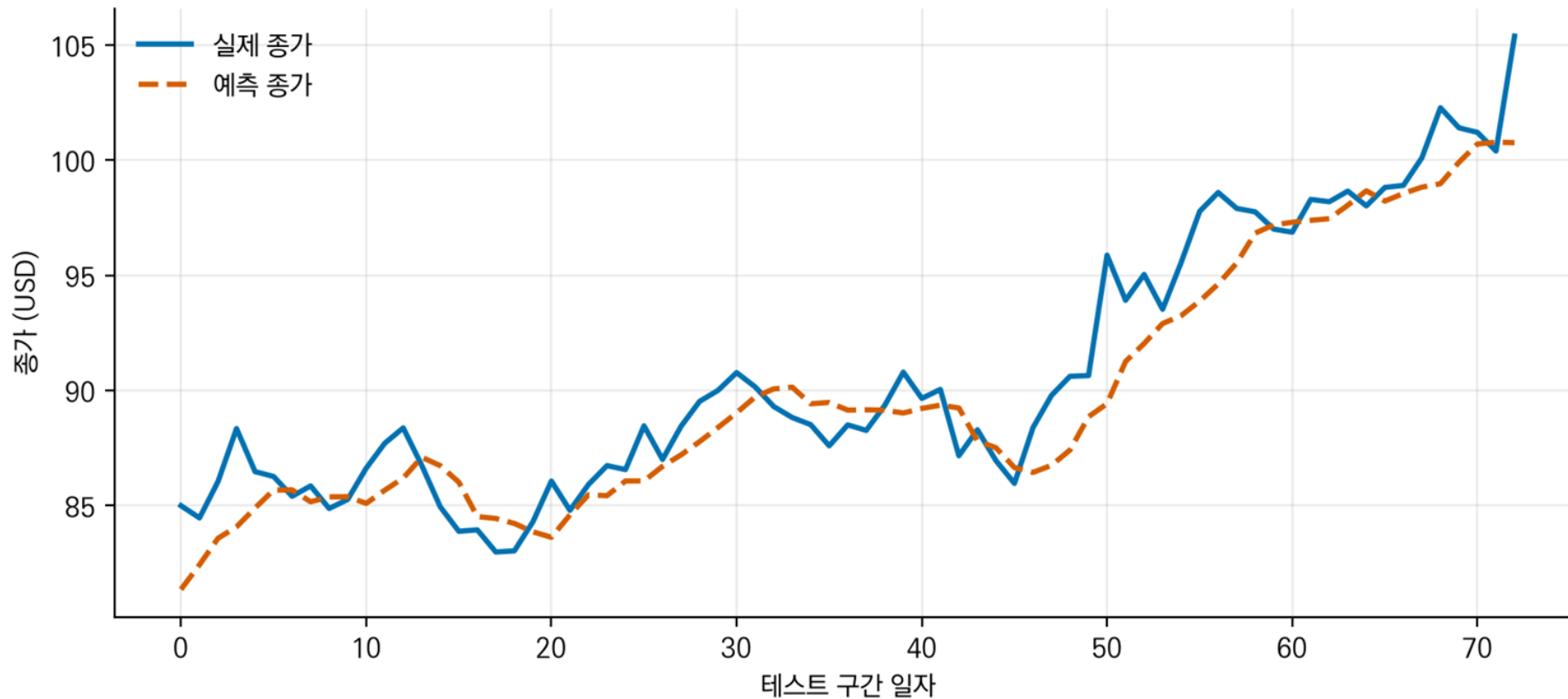
```
class LSTMRegressor(nn.Module):  
    def __init__(self, input_dim):  
        self.lstm = nn.LSTM(input_dim, 32, batch_first=True)  
        self.fc = nn.Linear(32, 1)  
    def forward(self, x):  
        out, _ = self.lstm(x, (h0, c0))  
        return self.fc(out[:, -1, :])  
  
X = [scaled[i:i+SEQ] for i in range(len(scaled)-SEQ)]  
loss_fn = nn.MSELoss()
```

정규화 → 슬라이딩 윈도우 → LSTM → 다음 값(회귀)
SBUX=4, sales=1
연속값 1개
다음 값 예측
슬라이딩 윈도우
회귀 → MSE

- MinMaxScaler로 0~1 정규화, 과거 N스텝 → 다음 값을 잘라내는 슬라이딩 윈도우.
- 분류와 차이: 출력 1개(연속값) + MSE + inverse_transform 으로 단위 복원.
- SBUX는 특징 4개(다변량), sales는 1개(단변량) — input_dim 만 다름.

실습③ SBUX 주가 예측 결과

SBUX 주가 예측 — 실제 vs 예측 (테스트 구간, RMSE 1.95 USD)



예측이 실제를 잘 따라가지만 '전날 값 근처'를 따르는 경향(한 박자 지연). 주가는 랜덤워크에 가까움.

Part 1
RNN

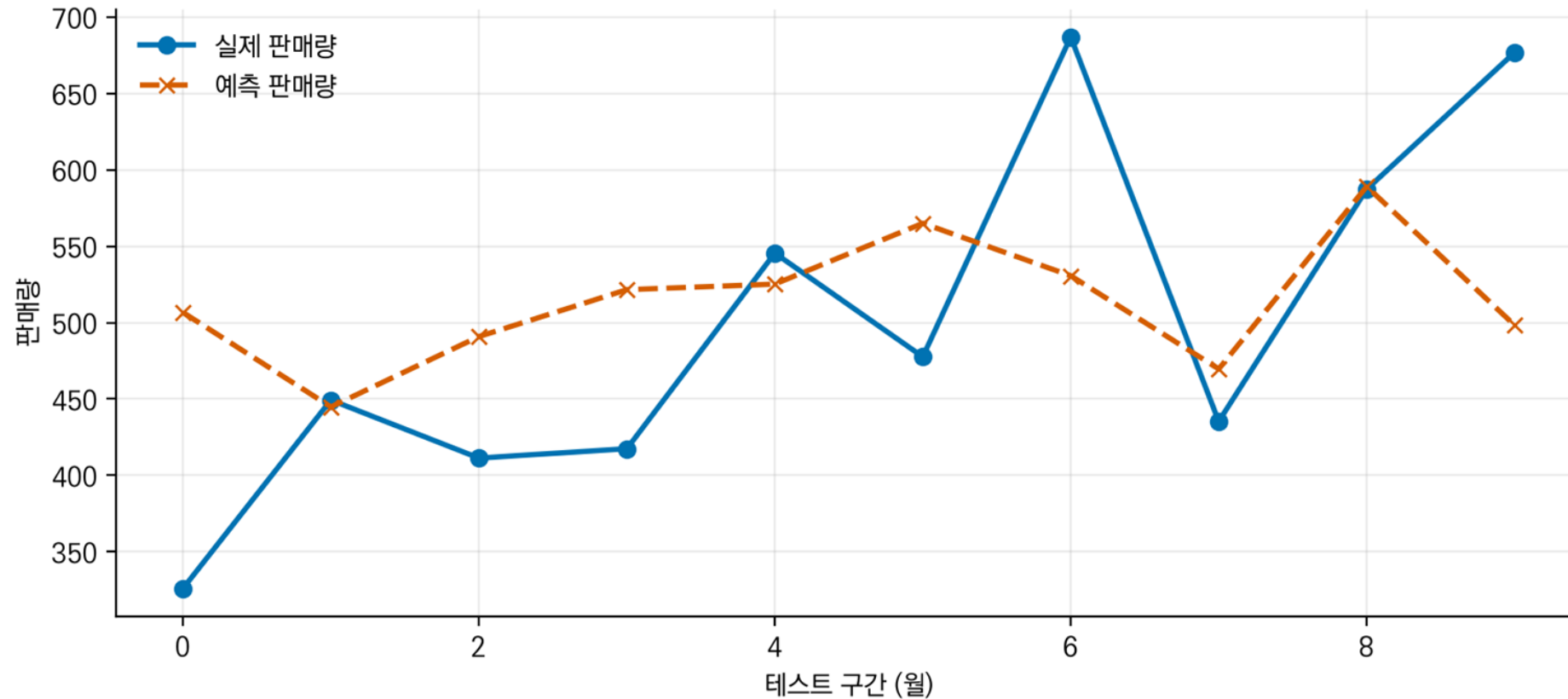
Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

실습④ 월별 판매량 예측 결과

월별 판매량 예측 — 실제 vs 예측 (테스트 구간, RMSE 107.5)



데이터 36개월로 적어 예측이 빗맞 — 데이터 양이 성능에 영향을 미침.

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

Part 1 RNN

```
# 같은 구조에서 셀만 RNN/LSTM/GRU 로 바꿔 각각 시연
self.rnn = nn.RNN(1, 16, batch_first=True) # ① 사인파 다음값(회귀)
self.lstm = nn.LSTM(1, 32, batch_first=True) # ② 첫 값 기억(장기기억)
self.gru = nn.GRU(1, 32, batch_first=True) # ③ 상승/하락 분류
```

Part 2 LSTM

```
out, _ = self.rnn(x) # RNN · GRU 는 출력만
out, (h, c) = self.lstm(x) # LSTM 만 (h, c) 튜플 반환
pred = self.fc(out[:, -1, :]) # 마지막 타임스텝 사용
```

Part 3 GRU

Part 4 실습 · 정리

- 세 모델을 최소 코드로 각각 시연 (데이터는 코드가 생성 → 업로드 불필요).
- 유일한 차이: nn.RNN / nn.LSTM / nn.GRU 한 줄, LSTM만 (h, c) 튜플 반환.
- 성능이 아니라 "각 모델이 무슨 일을 하는지"를 보여주는 용도.

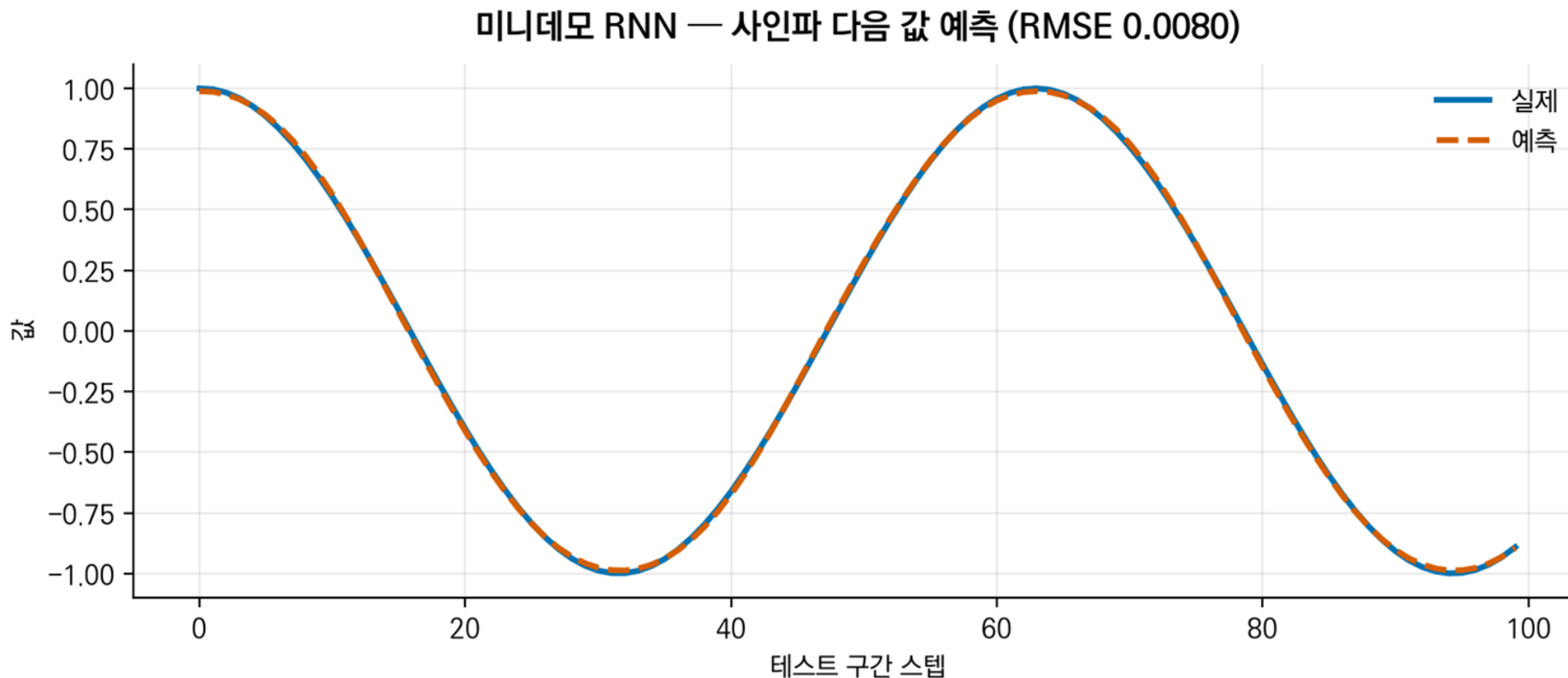
미니데모 — RNN 사인파 다음값 예측

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리



과거 20스텝으로 다음 값 예측. 예측·실제가 거의 완전히 겹침(RMSE 0.008) — 짧고 규칙적인 신호는 기본 RNN으로 충분.

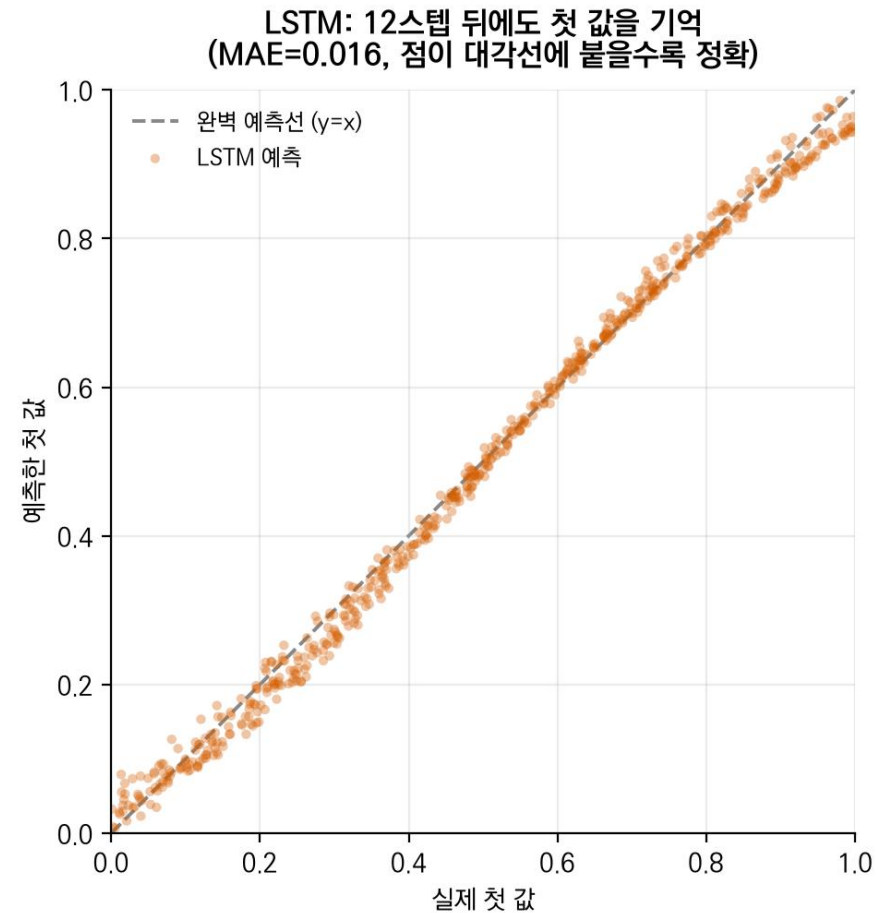
미니데모 — LSTM 기억력 (첫 값 복원)

Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리



12스텝 뒤에도 첫 값을 정확히 기억(MAE 0.016). 점이 대각선에 밀착할수록 정확.

미니데모 — GRU 추세 분류

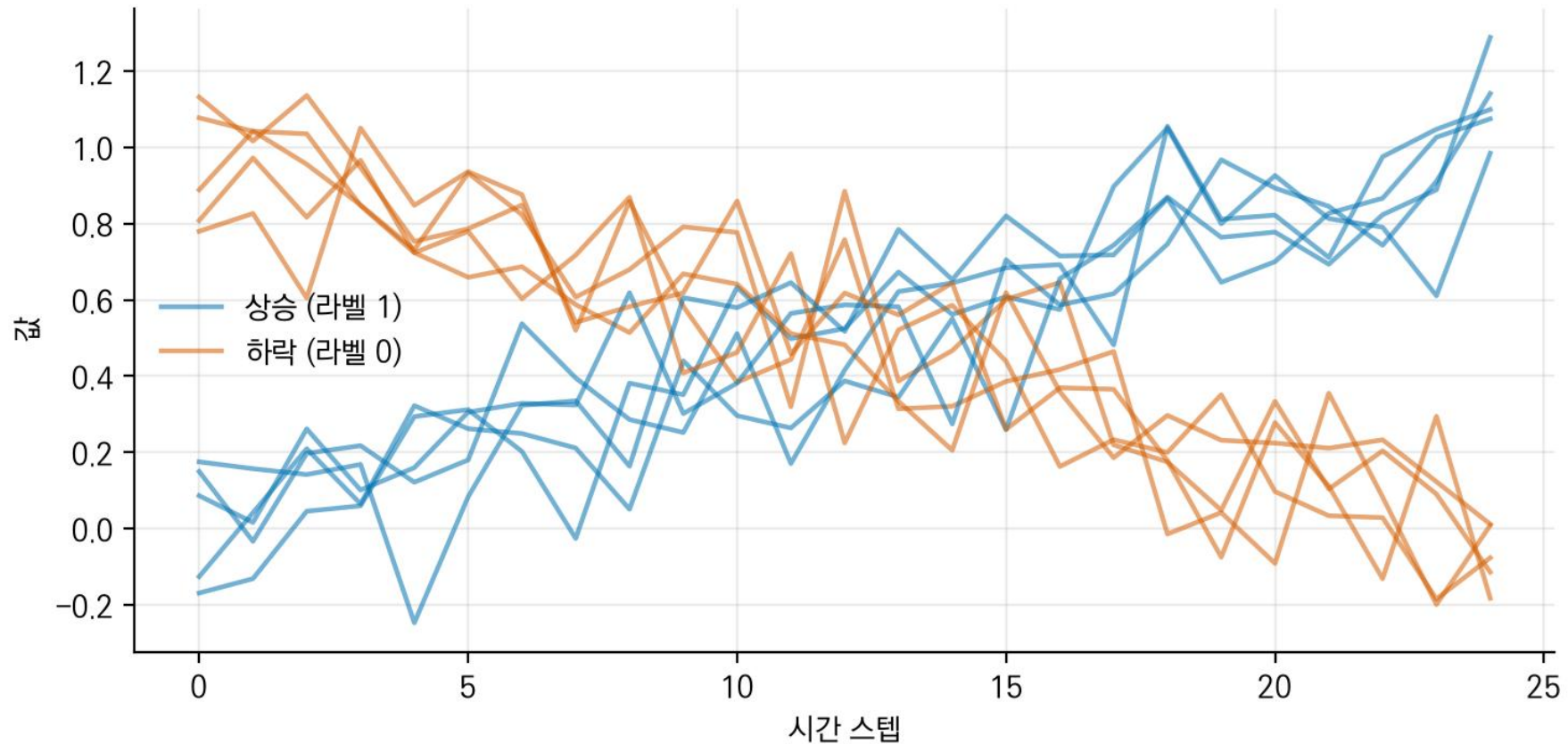
Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

GRU 과제: 노이즈 속 상승 vs 하락 추세 분류 (정확도 100%)



노이즈 속 상승/하락 추세를 100% 정확도로 분류.

미니데모 — LSTM vs GRU 파라미터 수

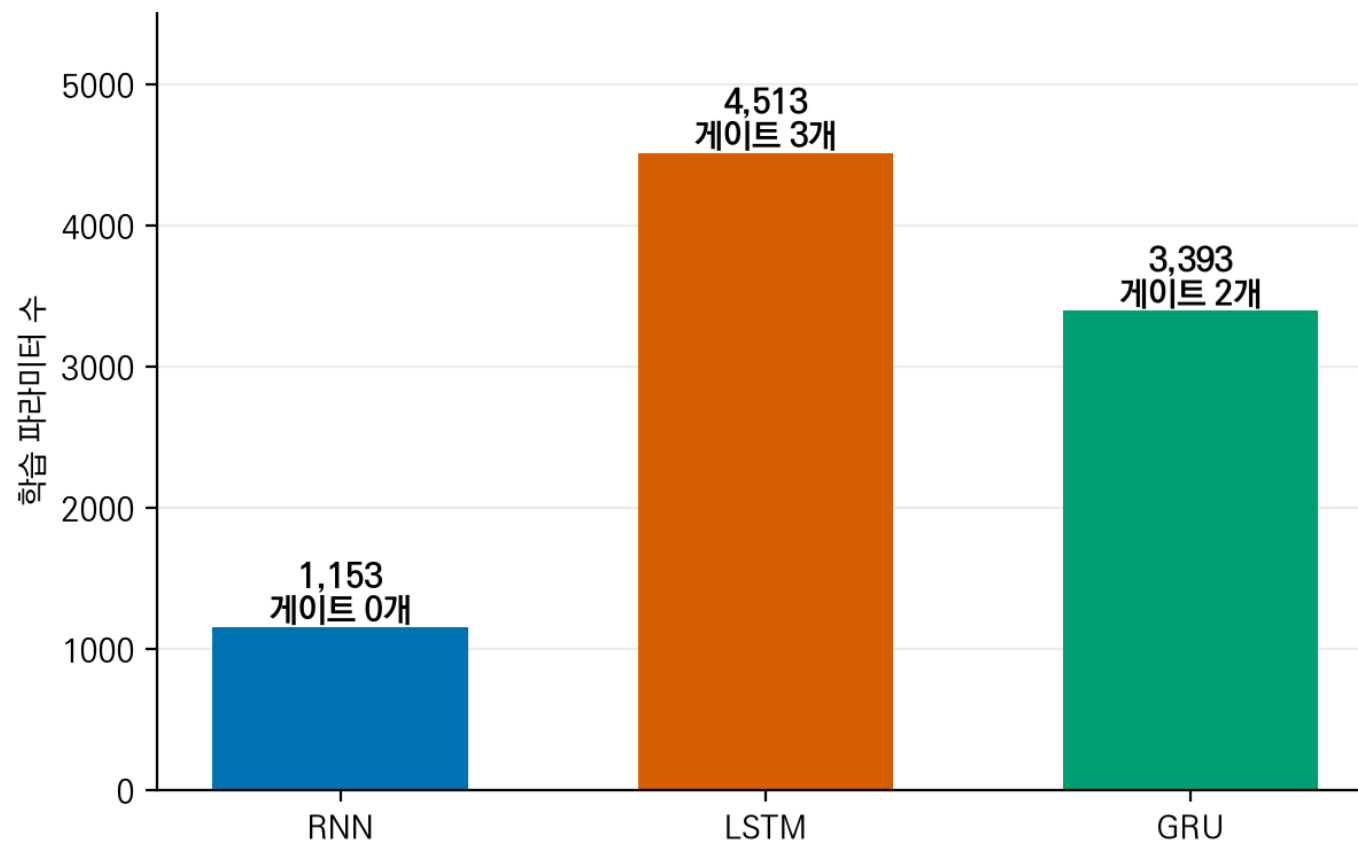
Part 1
RNN

Part 2
LSTM

Part 3
GRU

Part 4
실습 · 정리

게이트가 많을수록 파라미터가 많다 (hidden=32 기준)



GRU는 게이트가 하나 적어 LSTM의 약 75% 파라미터. 성능은 비슷, 더 가볍다.

Any Questions?

이름 안동균

소속 한양대학교 로봇공학과

이메일 ehdrbs123450@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

